

Scalable Sentiment Classification: Word2Vec Vectorization and Document Length Benchmarking

Part 1: Text Data Exploration

Detailed Summary

In part one, *"Text Data Exploration"* outlines the construction and evaluation of a natural language processing (NLP) machine learning pipeline designed to classify textual sentiment. The study expands upon prior personal work by shifting from traditional frequency-based text representations to dense, distributed word embeddings.

Data and Preprocessing Pipeline

The project utilized the Sentiment140 dataset sourced from Kaggle, which contains approximately 1.6 million public tweets.

- **Class Configuration:** Although the original documentation noted three sentiment states, exploratory data analysis revealed that the available dataset only contained two classes: negative (0) and positive (4). The task was therefore treated as a binary classification problem, and target metrics were re-mapped to 0 (negative) and 1 (positive).
- **Noise Reduction:** Metadata fields were discarded to retain only raw text and targets. Regular expressions (regex) were implemented to strip out URLs, user mentions (@username), excessive whitespace, punctuation, and special characters. All characters were converted to lowercase, empty rows were expunged, and the rows were split into training and testing partitions using a stratified layout to secure uniform class distributions.

Word2Vec Feature Extraction

To capture deep semantic relationships between words, the cleaned corpus was tokenized into individual words, and very short tokens were filtered out. The resulting text framework established a massive scale:

- **Total Documents:** Over 1 million records.
- **Total Tokens:** Approximately 13.5 million tokens.
- **Unique Vocabulary:** Over 400,000 unique words.

The feature representation model was built using the Word2Vec skip-gram architecture, selected for its proficiency in mapping less frequent yet contextually rich terms. The embedding dimension was pinned to 200. Because the classification algorithm requires fixed-length input spaces, each tweet was translated into a single document-level feature vector by calculating the mathematical average of all word embeddings present in that tweet.

Classification Results and Limitations

The averaged document vectors were fed directly into a multinomial Logistic Regression classifier equipped with class weighting to neutralize any remaining data imbalances.

- **Performance:** The baseline model achieved a consistent 75% overall accuracy on the unseen test set. Precision, recall, and F1-scores were highly balanced across both target categories, with the negative class scoring a 0.75 F1-score and the positive class scoring a 0.76 F1-score.

It was concluded that despite the informal and noisy nature of Twitter language, the pipeline successfully extracted meaningful semantic context. However, a core structural limitation was noted: the process of averaging word embeddings strips away text sequence ordering and syntax, which can lead to misclassifications in instances involving negation or complex word combinations. Future iterations will focus on tuning hyperparameters (such as window sizes and training epochs) and evaluating alternative frameworks like GloVe.

Part 2: Sentiment Classification Using Word2Vec and Logistic Regression

Detailed Summary

In part two, "*Sentiment Classification Using Word2Vec and Logistic Regression*" outlines an industrial and pipeline-focused evaluation of natural language processing techniques. The primary goal was to determine if Word2Vec text embeddings, combined with a standard Logistic Regression model, could capture textual sentiment accurately while maintaining structural stability across multiple training settings.

Dataset & Preprocessing Pipeline

The experiment utilized the IMDb Dataset of 50K Movie Reviews from Kaggle, containing 50,000 text blocks evenly balanced between positive and negative labels. Movie reviews were intentionally chosen over short-form text due to their richer contextual signals and descriptive length.

- **Data Cleansing:** To prepare raw text for tokenization, HTML tags (e.g.,
) and web entities were scrubbed, and text fields were converted to lowercase. Punctuation and URLs were filtered out, though internal word apostrophes were preserved to keep contractions intact. Empty rows were subsequently dropped.
- **Data Partitioning:** The processed corpus was divided into an 80% training set and a 20% testing set using a stratified split to preserve class equilibrium. Word2Vec models were strictly isolated to the training partition to avoid data leakage.

Modeling & Training Characteristics

The text representation phase employed a Word2Vec skip-gram architecture (sg=1), selected for its strength with large vocabularies and sentence-level semantic links. Baseline constraints featured a vector size of 200, a minimum word frequency threshold of 5, and a context window of 5 tokens. The training corpus established a robust baseline profile:

- Total Documents: ~40,000
- Total Tokens: ~7.3 million
- Unique Vocabulary: 108,977 words
- Average Length: 184 tokens per review

To construct classification inputs, variable-length reviews were mapped into fixed-length vectors by averaging the word embeddings of their constituent tokens. These served as features for the downstream Logistic Regression classifier.

Experimental Findings & Evaluation

- **The Epoch Paradox:** The Word2Vec framework was tested across 5, 10, and 20 epochs. While tracking graphics showed that training loss continued to drop across longer runs, downstream classification accuracy remained entirely stagnant at approximately 88%. Consequently, the 5-epoch run was crowned the most computationally efficient pipeline variation.
- **Classification Accuracy:** The optimized pipeline secured a test accuracy of 0.8817, with matching 0.88 metrics for precision, recall, and F1-scores. Confusion matrix analysis proved classification errors were distributed nearly evenly, signaling no class bias.
- **Stability Testing:** A stratified 5-fold cross-validation demonstrated high generalization stability, yielding a mean F1-score of 0.88 with a minimal standard deviation ($\sigma=0.004$).
- **Length Distributions:** A review length breakdown (short, medium, long, very long) confirmed the data was heavily dominated by medium and long texts. Averaging token embeddings over longer sequences naturally stabilized semantic density, yielding superior text vector profiles.

It was concluded that this pipeline vastly outperformed the author's previous short-form Twitter text project (*Text Data Exploration*), which capped at 75% accuracy due to slang and minimal context. Future work recommendations include integrating GloVe or transformer-based embeddings and conducting an audit of misclassified text blocks containing sarcasm or ambiguous sentiment.